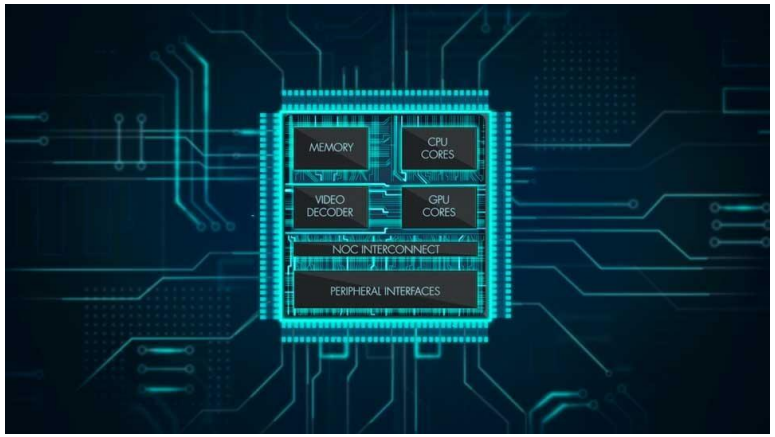
# Digital System Design

# Basic Practice 6





Date : 2026. 04. 30.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

# Contents


- Problem 1: FSM and Controller
- Problem 2: Padding
- Problem 3: Multi-Layer and ReLU

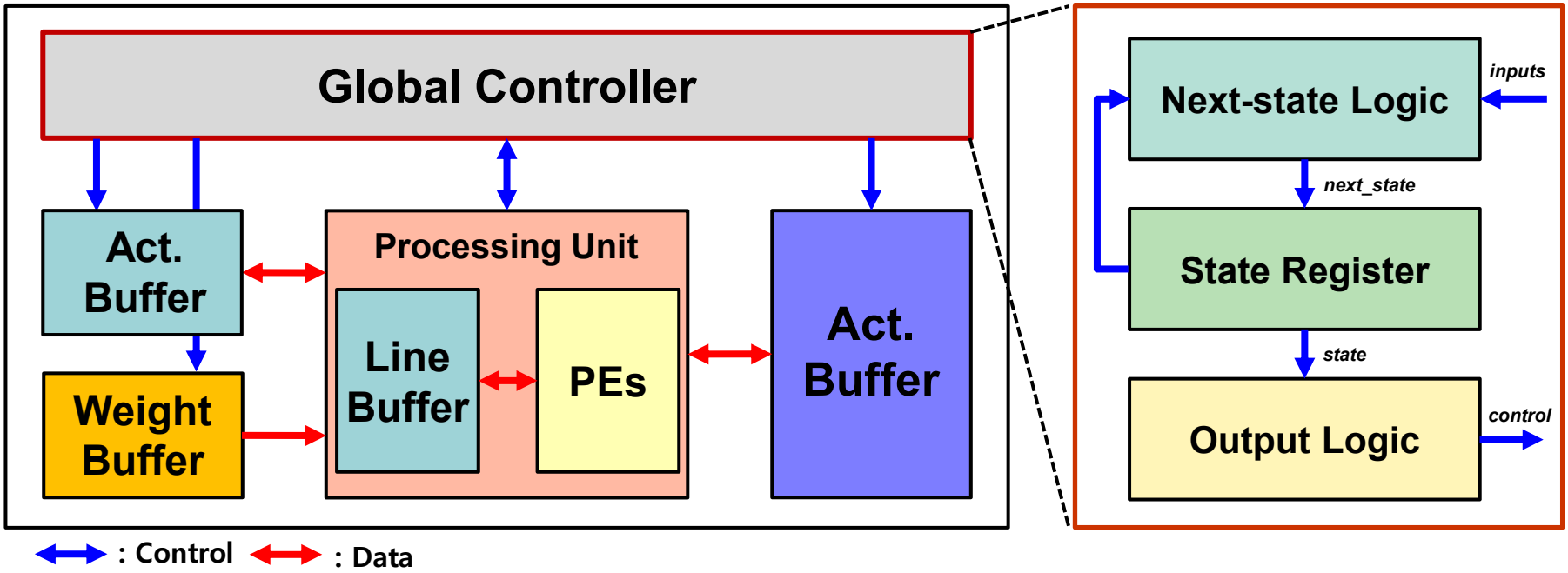
# Contents


- Problem 1: FSM and Controller
- Problem 2: Padding
- Problem 3: Multi-Layer and ReLU

# Prob 1. FSM and Controller(1)

## Controller

- Controller based on **Basic Practice 5, Problem 1.**
- Testbench will now only give *i_clk*, *rstn*, and *i_start* signals.

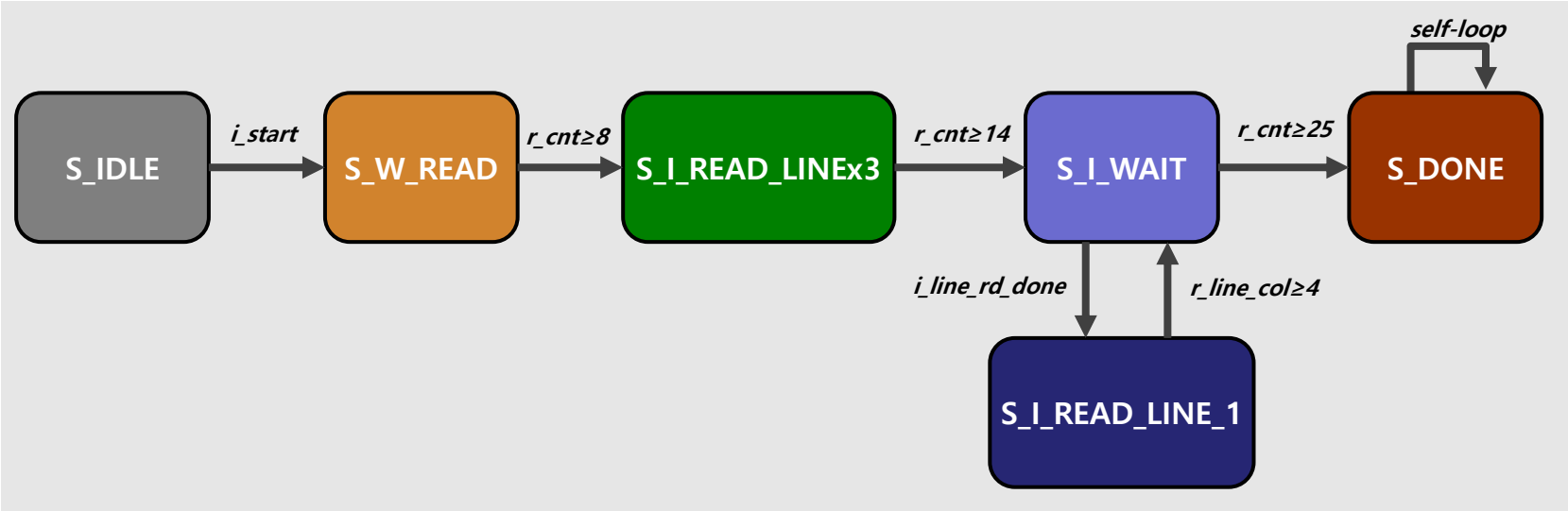


# Prob 1. FSM and Controller(2)



## FSM

- **S_IDLE(0):** After system reset, wait for the operation start signal (i_start).
- **S_W_READ(1):** Read the 3x3 filter weights and load them as initial values into the PU.
- **S_I_READ_LINEx3(2):** Load the first 3 rows (15 pixels) of input data into the buffer for the initial convolution.
- **S_I_WAIT(3):** Wait for the row (line buffer) operation to complete and check how much of the input has been processed.
- **S_I_READ_LINE_1(4):** Read one additional row (5 pixels) to update the buffer for window sliding.
- **S_DONE(5):** Signal completion of the entire operation (o_done) and hold the terminal state.

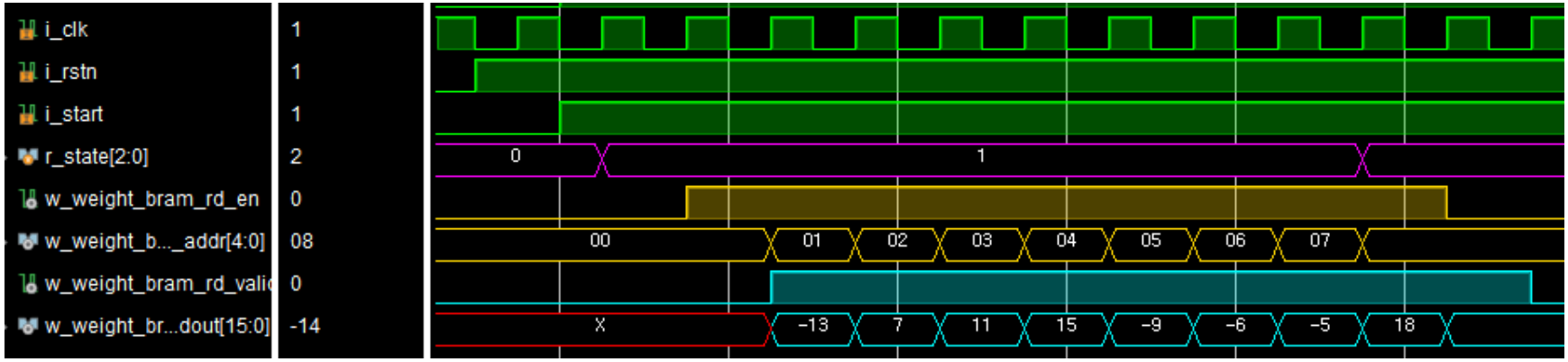


# Prob 1. FSM and Controller(3)

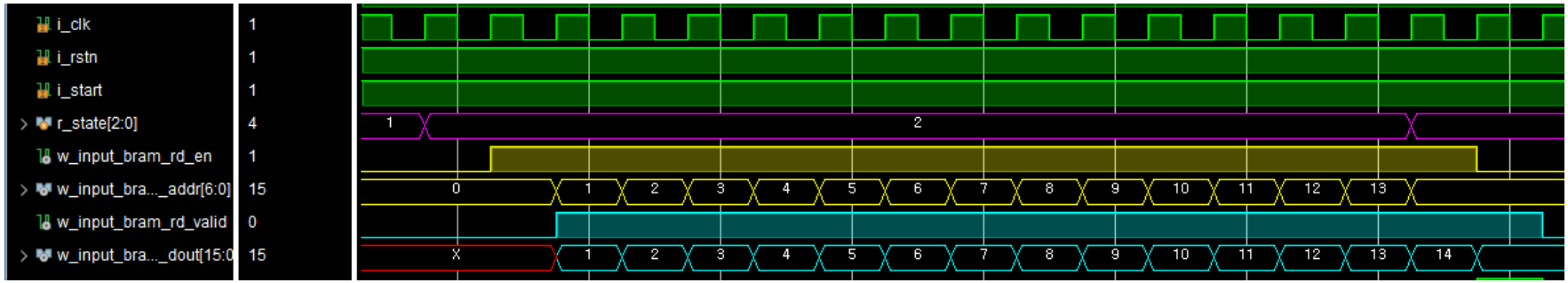


## Waveform

- **S_W_READ(1):** Read the 3x3 filter weights and load them as initial values into the PU.




- **S_I_READ_LINEx3(2):** Load the first 3 rows (15 pixels) of input data into the buffer for the initial convolution.

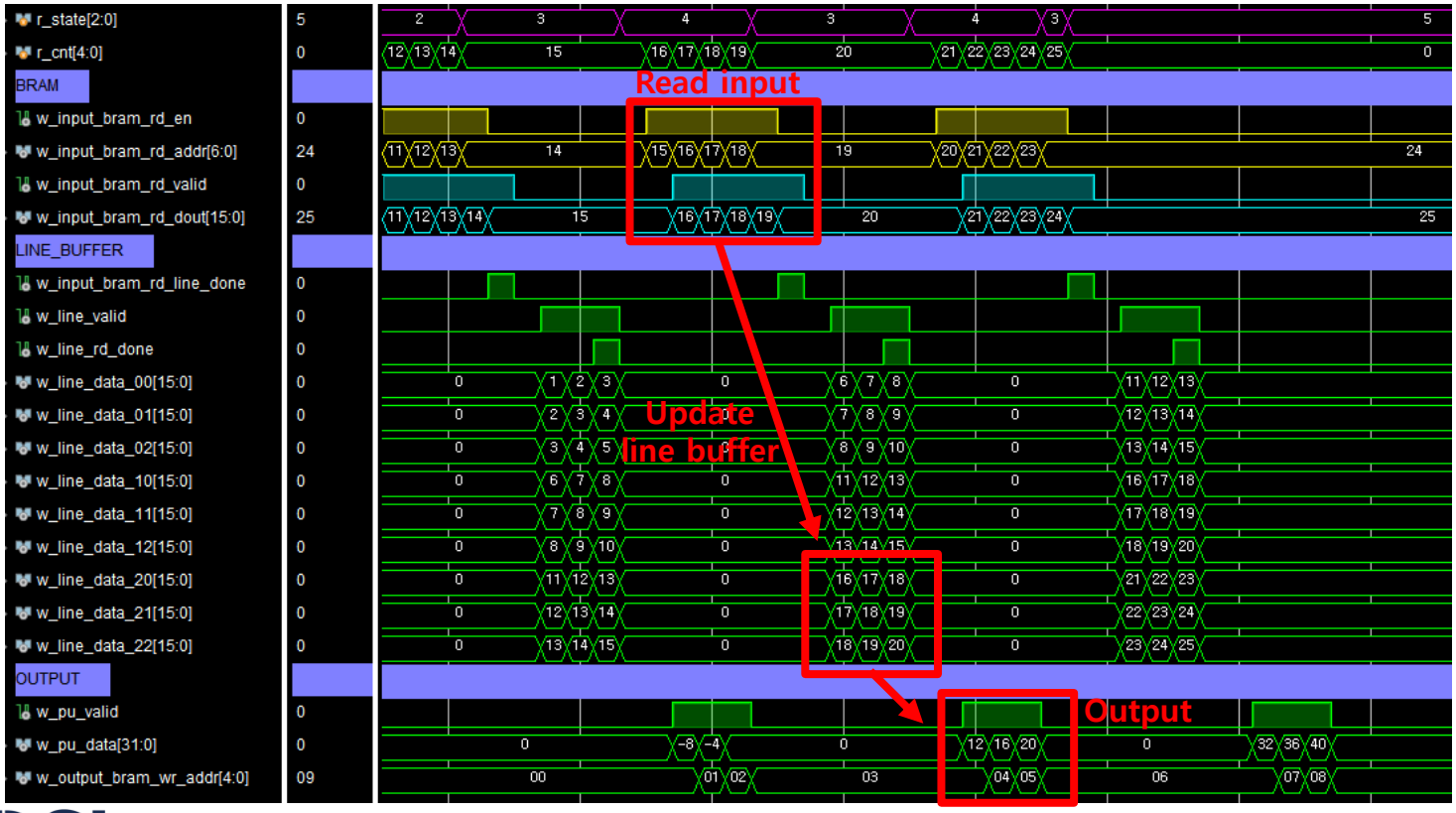


# Prob 1. FSM and Controller(4)



## Waveform

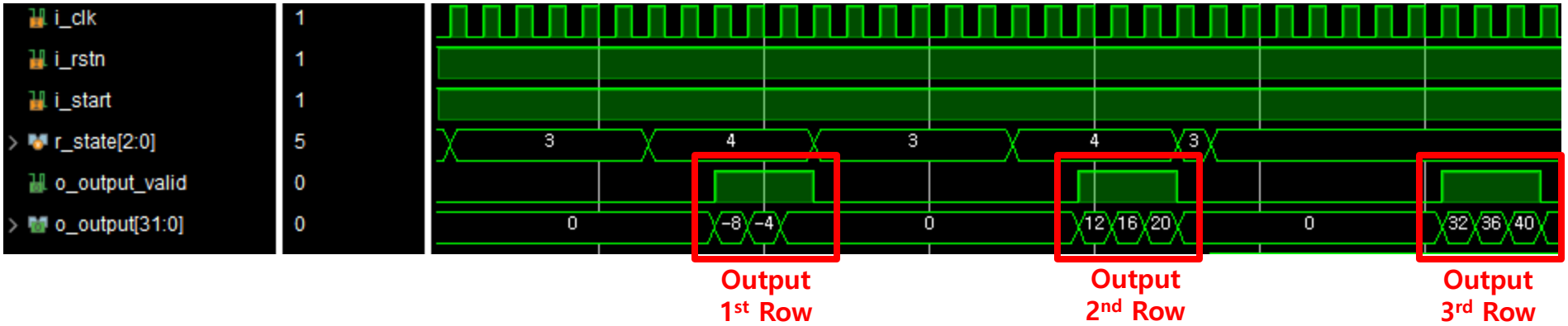
- **S_I_WAIT(3):** Wait for the row (line buffer) operation to complete and check how much of the input has been processed.
- **S_I_READ_LINE_1(4):** Read one additional row (5 pixels) to update the buffer for window sliding.
- **S_DONE(5):** Signal completion of the entire operation (o_done) and hold the terminal state.



# Prob 1. FSM and Controller(5)

## Waveform

- Close-up on output.




- Visualization of input, weight, and output matrix.

**Input**

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
| 6 | 7 | 8 | 9 | 10 |
| 11 | 12 | 13 | 14 | 15 |
| 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 |

×

**Weight**

| | | |
|---|---|---|
| -13 | 7 | 11 |
| 15 | -9 | -6 |
| -5 | 18 | -14 |

=

**Output**

| | | |
|---|---|---|
| -8 | -4 | 0 |
| 12 | 16 | 20 |
| 32 | 36 | 40 |

# Contents


- Problem 1: FSM and Controller
- Problem 2: Padding
- Problem 3: Multi-Layer and ReLU

# Prob 2. Padding(1)

## What is Padding?

- **Padding** is the process of adding extra pixels (usually zeros) around the border of an input image before applying a convolution.
- It helps **preserve the spatial size** of the feature map and **prevents edge information from being lost** too quickly.
- Padding allows the filter to cover border pixels more effectively, improving feature extraction near the image boundaries.

**Without padding:**

3x

3x

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
| 6 | 7 | 8 | 9 | 10 |
| 11 | 12 | 13 | 14 | 15 |
| 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 |

$\times$

| | | |
|---|---|---|
| -13 | 7 | 11 |
| 15 | -9 | -6 |
| -5 | 18 | -14 |

$=$

| | | |
|---|---|---|
| -8 | -4 | 0 |
| 12 | 16 | 20 |
| 32 | 36 | 40 |

**With padding:**

5x

5x

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 2 | 3 | 4 | 5 | 0 |
| 0 | 6 | 7 | 8 | 9 | 10 | 0 |
| 0 | 11 | 12 | 13 | 14 | 15 | 0 |
| 0 | 16 | 17 | 18 | 19 | 20 | 0 |
| 0 | 21 | 22 | 23 | 24 | 25 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$\times$

| | | |
|---|---|---|
| -13 | 7 | 11 |
| 15 | -9 | -6 |
| -5 | 18 | -14 |

$=$

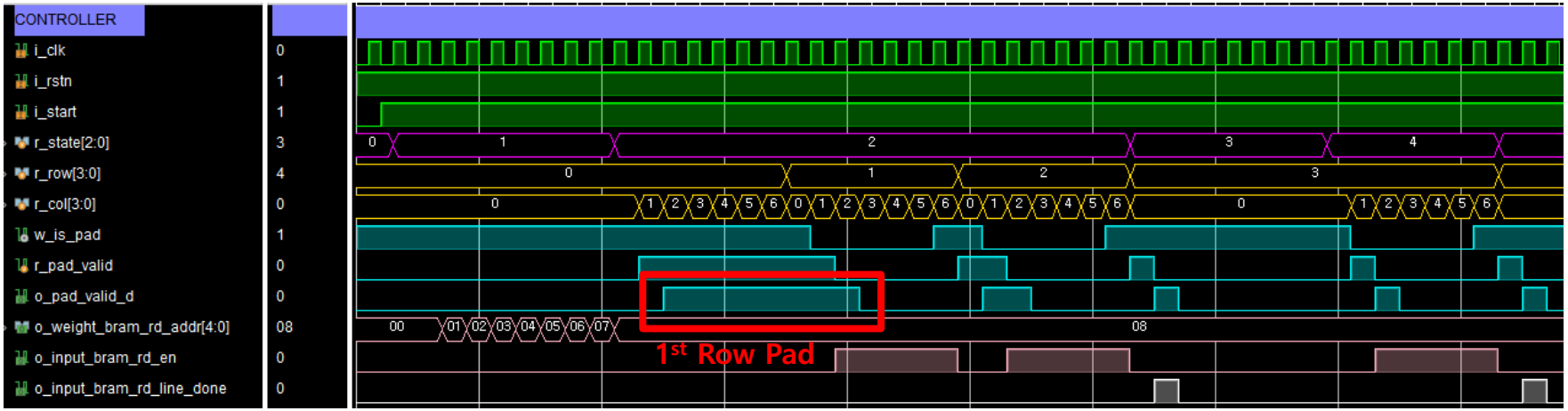
| | | | | |
|---|---|---|---|---|
| -11 | -37 | -38 | -39 | 150 |
| -37 | -8 | -4 | 0 | 228 |
| -2 | 12 | 16 | 20 | 293 |
| 33 | 32 | 36 | 40 | 358 |
| -22 | 88 | 93 | 98 | 28 |

# Prob 2. Padding(2)

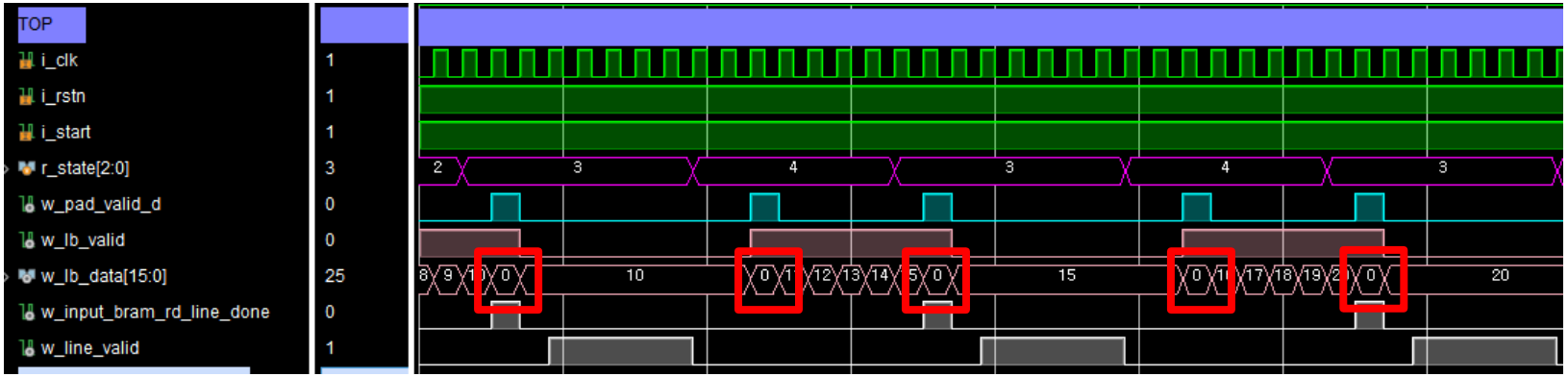


## Waveform

- Controller




- Top

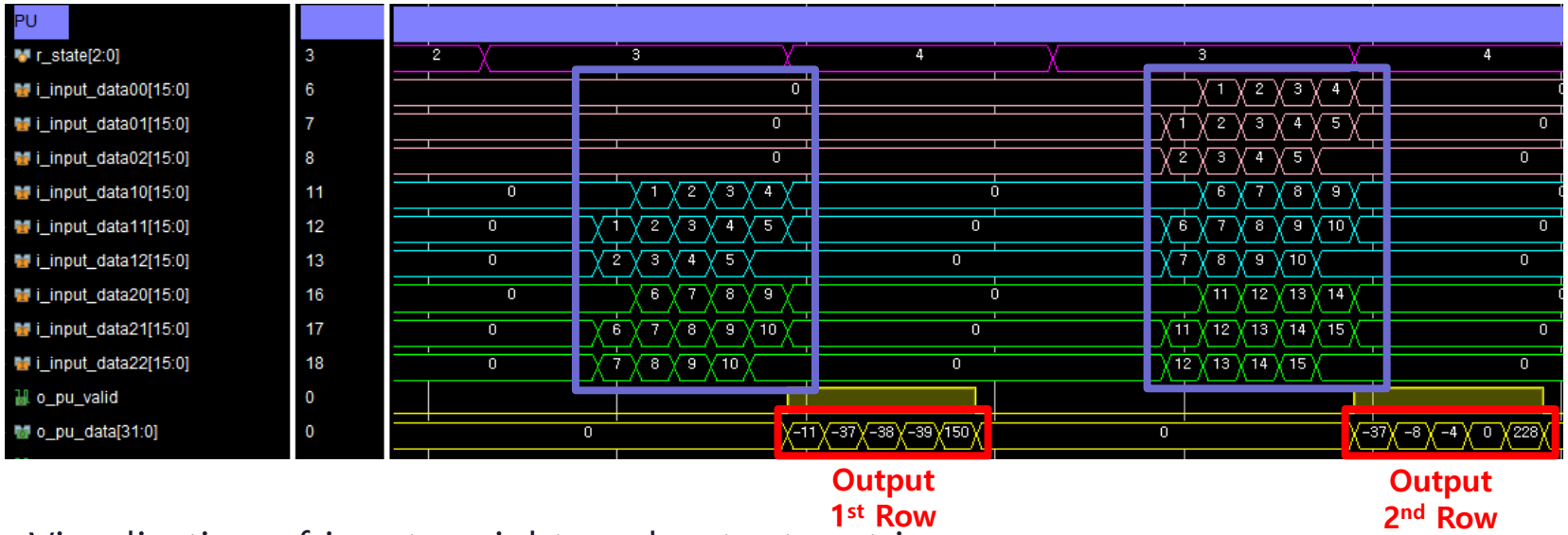



: Zero Padding

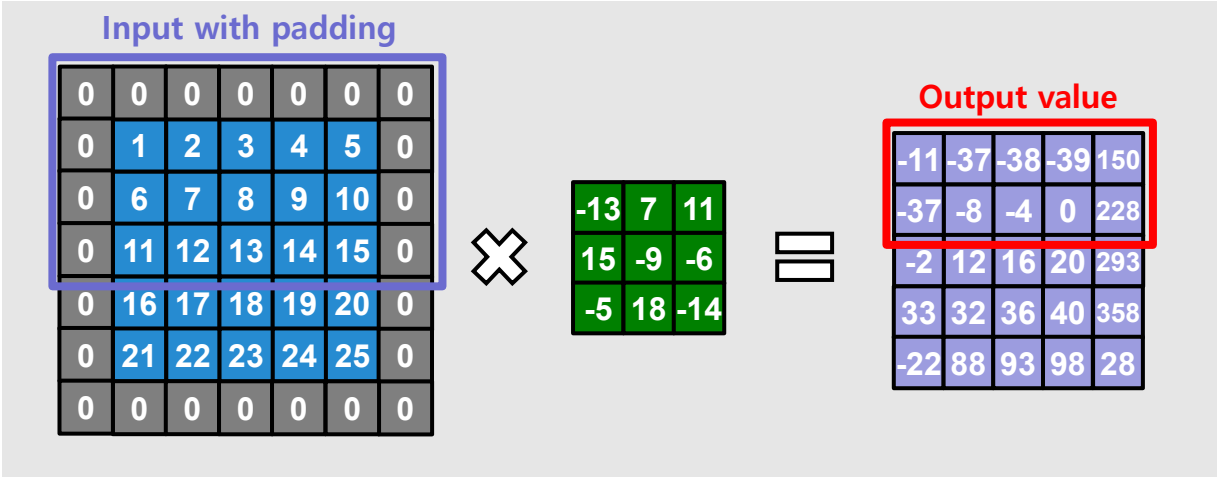
# Prob 2. Padding(3)

## Waveform

- PU




- Visualization of input, weight, and output matrix.



# Contents


- Problem 1: FSM and Controller
- Problem 2: Padding
- Problem 3: Multi-Layer and ReLU

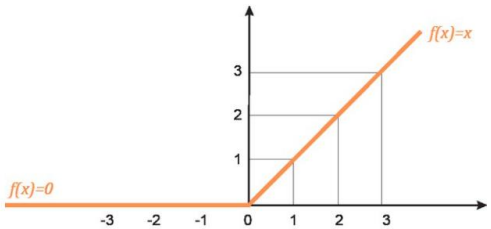
# Prob 3. ReLU

## What is ReLU?

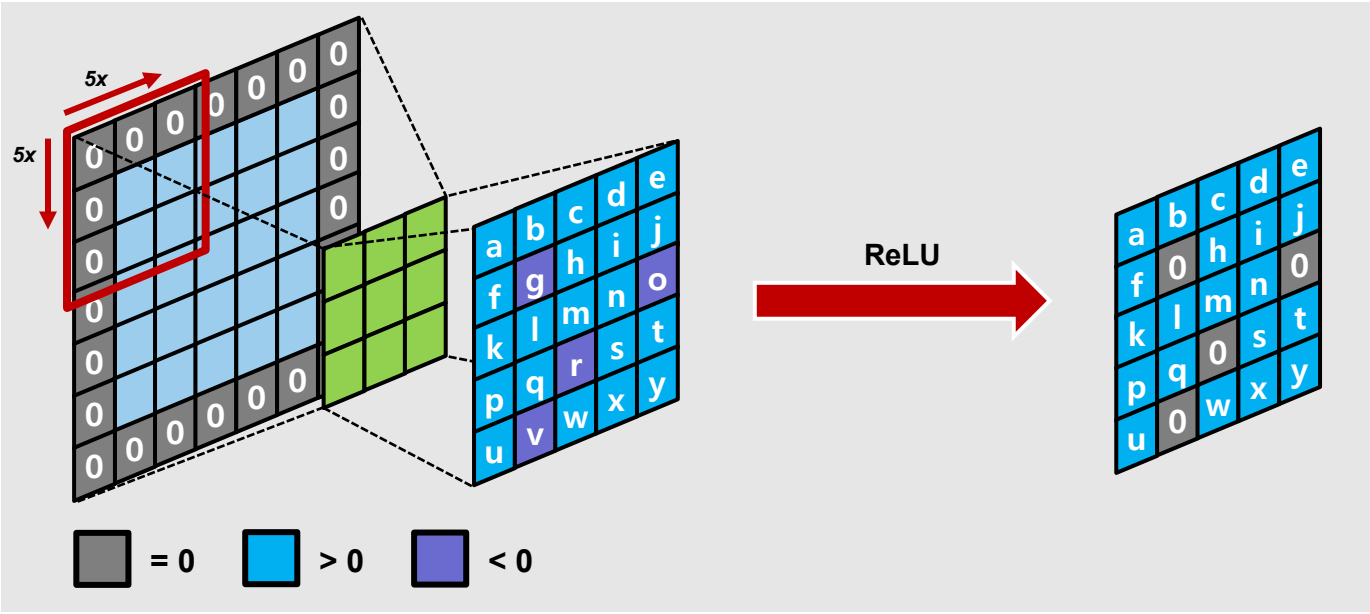
- **Re**ctified **L**inear **U**nit
- **Activation function**: Function applied to layer's output value, deciding how much information is passed to the next layer.
- Provides **nonlinearity**, allows neural network to learn complex patterns.

$$f(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

*<ReLU Equation>*




*<ReLU Graph>*




*<ReLU Example>*

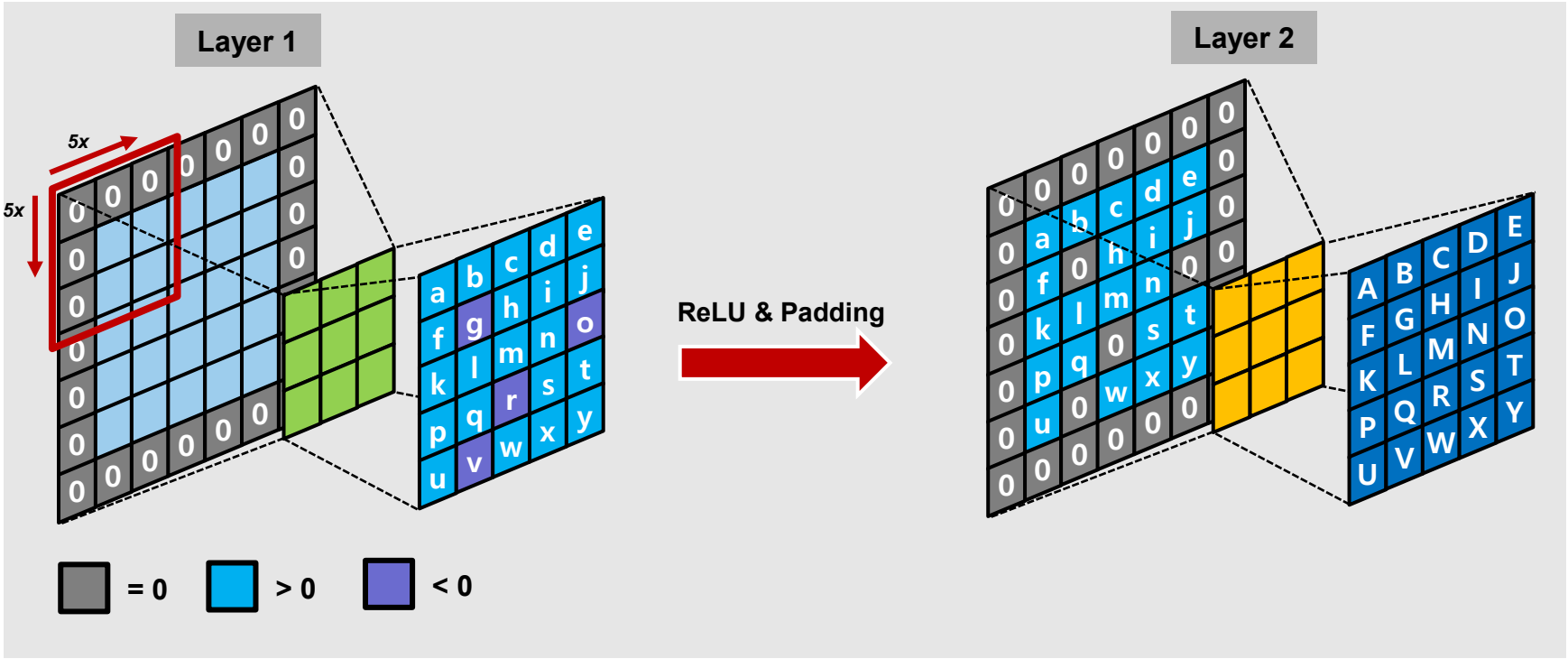


IDSL
Intelligent Digital Systems Design Lab.
SEOULTECH

# Prob 3. ReLU and Padding

## Multi Layer CNN naïve example

- Applying ReLU and padding.



# Prob 3. Multi-Layer and ReLU(1)

## Multi-Layer Structure problem guide

**Layer 1**

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 2 | 3 | 4 | 5 | 0 |
| 0 | 6 | 7 | 8 | 9 | 10 | 0 |
| 0 | 11 | 12 | 13 | 14 | 15 | 0 |
| 0 | 16 | 17 | 18 | 19 | 20 | 0 |
| 0 | 21 | 22 | 23 | 24 | 25 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$\times$

| | | |
|---|---|---|
| -13 | 7 | 11 |
| 15 | -9 | -6 |
| -5 | 18 | -14 |

$=$

| | | | | |
|---|---|---|---|---|
| -11 | -37 | -38 | -39 | 150 |
| -37 | -8 | -4 | 0 | 228 |
| -2 | 12 | 16 | 20 | 293 |
| 33 | 32 | 36 | 40 | 358 |
| -22 | 88 | 93 | 98 | 28 |

ReLU & Padding

**Layer 2**

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 150 | 0 |
| 0 | 0 | 0 | 0 | 0 | 228 | 0 |
| 0 | 0 | 12 | 16 | 20 | 293 | 0 |
| 0 | 33 | 32 | 36 | 40 | 358 | 0 |
| 0 | 0 | 88 | 93 | 98 | 28 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$\times$

| | | |
|---|---|---|
| 3 | 7 | -6 |
| -11 | 5 | 12 |
| 8 | -9 | 4 |

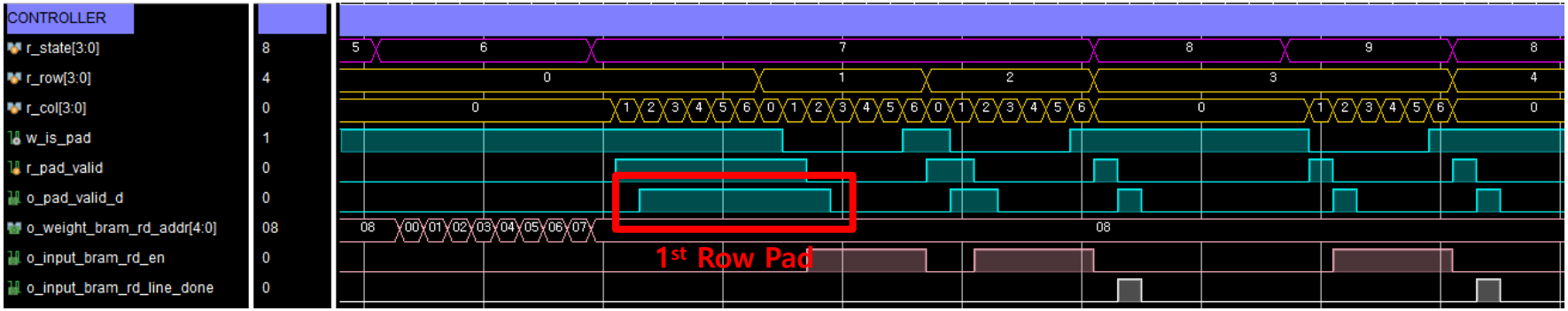
$=$

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | 2712 | -1302 |
| 48 | -44 | 32 | 2956 | -287 |
| -25 | 372 | 280 | 3432 | -61 |
| 829 | -203 | 595 | 2504 | 3993 |
| 1095 | 1663 | 781 | -1957 | 1688 |

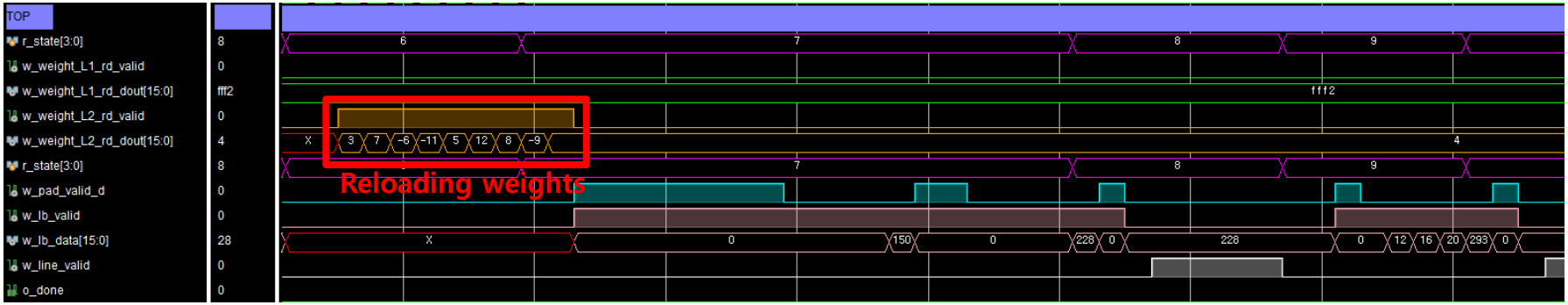
# Prob 3. Multi-Layer and ReLU(2)

## Waveform: similar waveform as problem 2

- Controller



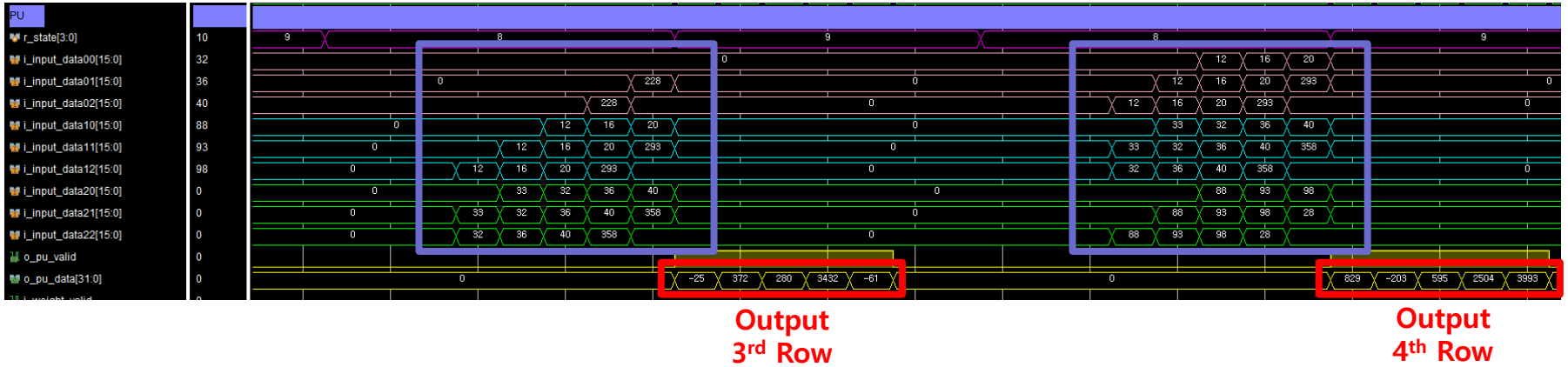

- Top



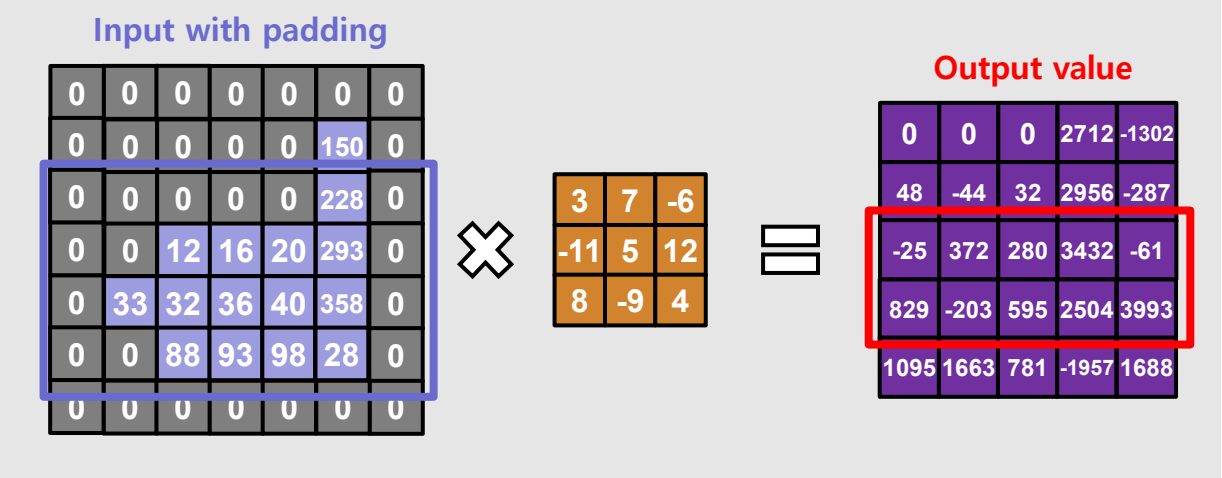
# Prob 3. Multi-Layer and ReLU(3)

## Waveform: similar waveform as problem 2

- PU




- Visualization of input, weight, and output matrix.



# Thank You!